```
            "field": "author.keyword"
          },
          {
            "field": "title.keyword"
          }
        ]
      }
    }
  }
}
```

As you can see, `multi_terms` accepts a set of terms. In the example, we expect Elasticsearch to return a book count using `author/title` keys. The response indicates that we were able to retrieve this information:

```
{
  "key" : [
    "Adam Scott",
    "JavaScript Everywhere"
  ],
  "key_as_string" : "Adam Scott|JavaScript Everywhere",
  "doc_count" : 1
},
{
  "key" : [
    "Al Sweigart",
    "Automate The Boring Stuff With Python"
   ],
   "key_as_string" : "Al Sweigart|Automate The Boring Stuff With Python",
   "doc_count" : 1
},
...
```

This response shows two variations of the `key` representation: as a set of fields (both `author` and `title`) and as a string (`key_as_string`), which simply stitches together the fields with a pipe (`|`) delimiter. The `doc_count` indicates the number of documents (books) in the index for that key.

If you are curious, rerun the query in listing 13.18, this time using the tags and the title as terms. You should get multiple books under the same tags (the code is available with the book's files).

Before we discuss the third type of aggregations—pipeline aggregations—we need to understand the concept of parent and sibling aggregations. These form the basis of pipeline aggregations. We discuss parent and sibling aggregations in the next section and then jump to pipeline aggregations.

13.4 Parent and sibling aggregations

Broadly speaking, we can group aggregations into two types: *parent* and *sibling* aggregations. You may find them a bit confusing, so let's see what they are and how they can be used.